



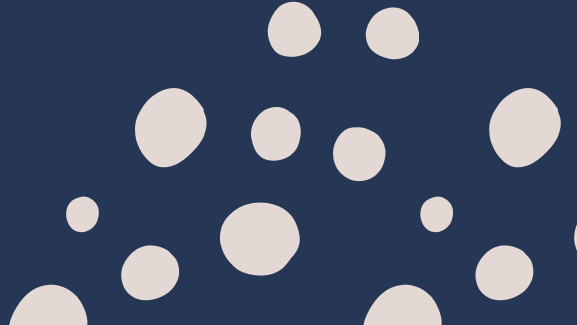
CODIGOS DE PRODUCTOS, ORDENAMIENTOS Y BUSQUEDAS DDYA



Problema

Una tienda maneja una lista con los códigos numéricos de sus productos. Como los productos se registran a medida que llegan, los códigos inicialmente se encuentran desordenados.

Se debe poder:

- Registrar una cantidad n de productos.
 - Ordenar sus códigos utilizando Merge Sort.
 - Buscar un código específico utilizando Búsqueda Binaria.
- 

Requerimientos

- RQFO1 Leer cantidad de productos n.
 - RQFO2 Leer el codigo de cada producto.
 - RQFO3 Muestra el orden de los codigos originales.
 - RQFO4: Ordenar los codigos de los productos de forma ascendente .
 - RQFO5: Leer el codigo del producto a buscar.
 - RQFO6: Mostrar la pocision del pructo si esta en la lista.
-
- RQNFO1 El sistema debe permitir ingresar una cantidad n de productos variables.
 - RQNFO2 El sistema no debe alterar los codigos de los productos .
 - RQNFO3 El sistema debe devolver una lista con los codigos de los productos ordenados.
 - RQNFO4 El sistema debe permitir ingresar un codigo de producto para buscar.
 - RQNFO5 El sistema debe devolver la pocision del codigo del producto, si no se encuentra en la lista devuelve un mensaje.

Historias de usuario

- Como usuario se debe tener la posibilidad de ingresar una cantidad n de productos, en base a la cantidad registrada se debe ingresar el código de cada producto y poder visualizar el orden inicial de los códigos.
- Como usuario se quiere visualizar el orden de los códigos de forma ascendente en un arreglo, para facilitar la búsqueda y visualización.
- Como usuario se debe tener la posibilidad de ingresar el código de un producto para realizar su búsqueda en el arreglo, además de visualizar su posición, si se encuentra en el arreglo, de lo contrario se debe recibir un mensaje que informe que el producto no se encuentra en la lista

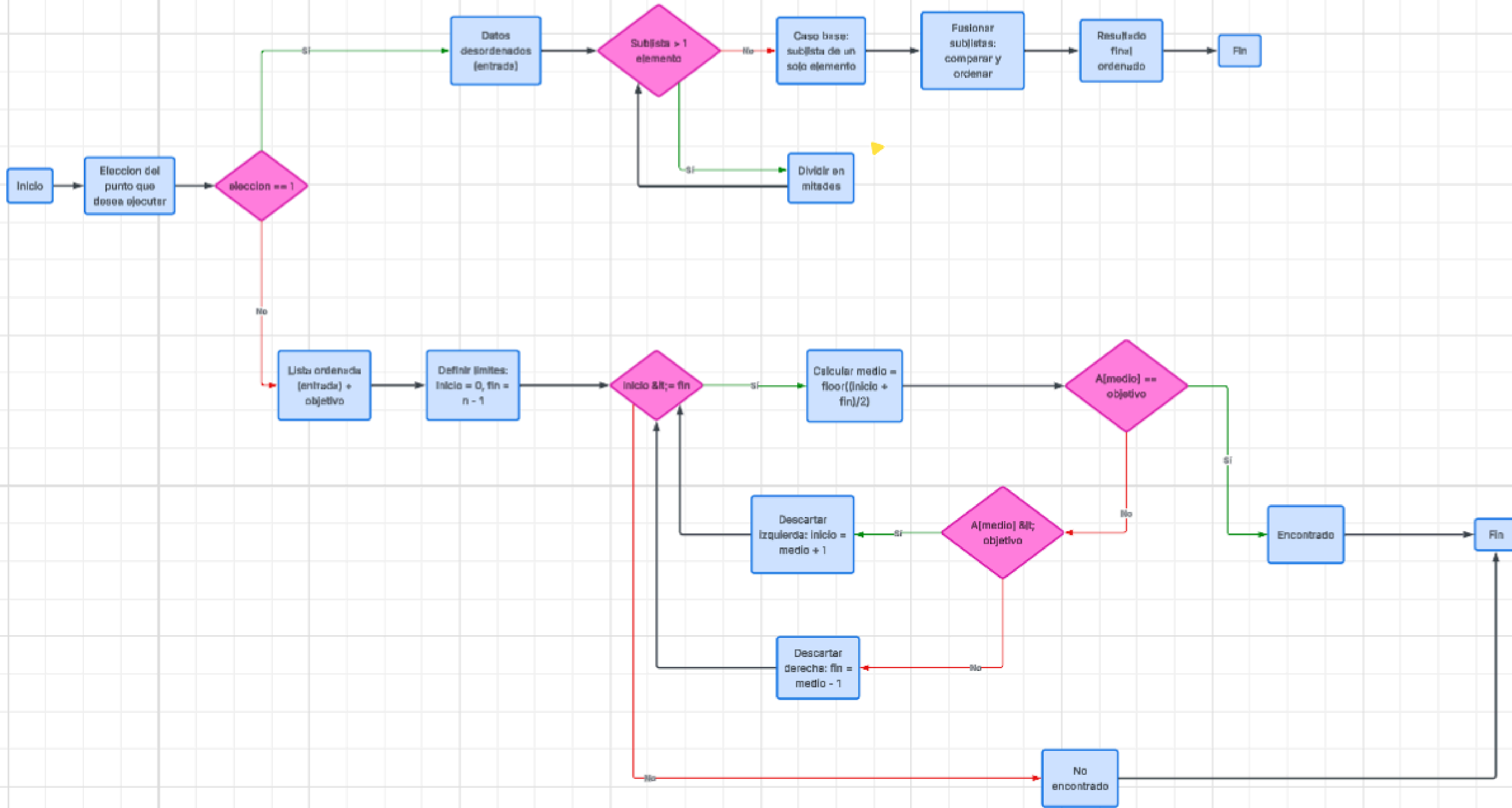
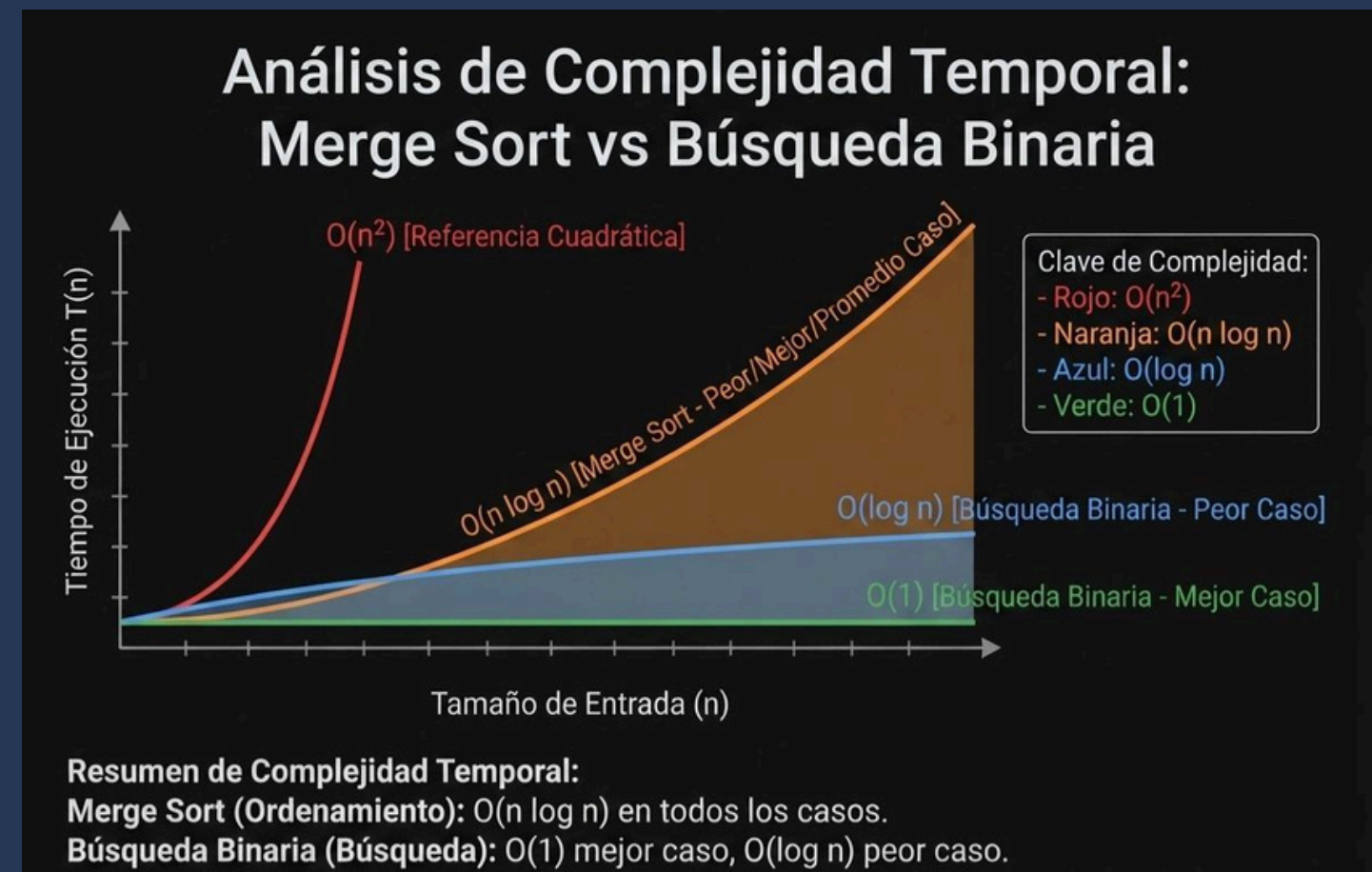


Diagrama de casos y secuencia



Análisis de Complejidad

- Merge Sort: Aplica Divide y Vencerás. Divide la lista en mitades ($\log_2 n$) y las fusiona linealmente ($O(n)$), garantizando un rendimiento estable.
- Búsqueda Binaria: Requiere la lista ordenada. Descarta el 50% de los datos en cada paso comparando con el centro, reduciendo drásticamente las operaciones.



```

def merge(tab, p, q, r):
    L = tab[p:q+1]
    R = tab[q+1:r+1]

    i = 0
    j = 0

    for k in range(p, r + 1):
        if i < len(L) and (j >= len(R) or L[i] <= R[j]):
            tab[k] = L[i]
            i = i + 1
        else:
            tab[k] = R[j]
            j = j + 1

def merge_sort(tab, p, r):
    if p < r:
        q = (p + r) // 2
        merge_sort(tab, p, q)
        merge_sort(tab, q + 1, r)

        merge(tab, p, q, r)

```

```

def binary_search_desc(tab, x):
    while lo <= hi:
        mid = (lo + hi) // 2
        if tab[mid] == x:
            return mid
        elif tab[mid] < x:
            lo = mid + 1
        else:
            hi = mid - 1
    return None

def main():
    n = int(input("Cantidad de productos (Valor NUMERICO entero): "))
    if n < 0:
        print("Valor no Valido")
    else:
        print("Valor valido, Valor guardado")
    tab = []

    for i in range(n):
        cod = int(input("CODIGO NUMERICO del producto: "))
        tab.append(cod)
    merge_sort(tab, 0, len(tab) - 1)

    print("Lista ordenada de menor a mayor:")
    print(tab)

    x = int(input("Ingresa el valor a buscar: "))
    res = binary_search_desc(tab, x)
    print(f"El codigo del producto se encuentra en la posicion {res} de la lista")

```

main()